FIWARE Intro

Gestión de Datos en Entornos Inteligentes (XDEI)

Contenido

- **REST Intro**
- **Tutoriales FIWARE NGSI v2**
  - *Getting started with NGSI v2*
  - *Entity Relationships*
  - *CRUD Operations*
  - *Accessing Context*
  - *Context Providers*
  - *Subscriptions*

REST APIs

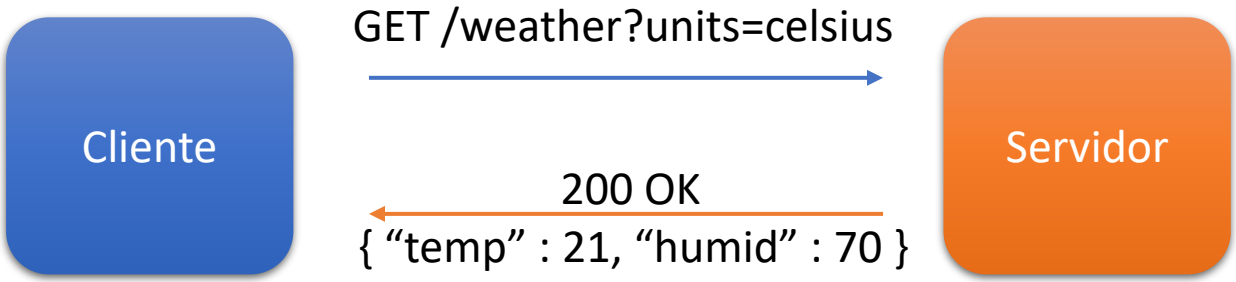
FIWARE Intro

REST APIs

- **M2M APIs**
  - Protocolos que permiten invocar **funciones o procedimientos remotos** (i.e. en otras máquinas)
  - Generalización a Internet del concepto de librería de programación
- **REST**
  - *REpresentational State Transfer*
  - **Alternativa ligera a protocolos API** basados en XML como WDSL sobre SOAP

- Ejemplos: Twitter, Facebook, Google Maps…

REST – Modelo

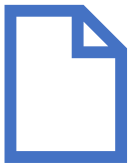


- Cliente – Servidor **HTTP**
- **Cliente envía**
  - **Método HTTP** (GET, POST, PUT, PATCH, DELETE)
  - **URL**: describe el objeto sobre el que actuar
  - **Cabeceras** con opciones (ej.: *Content-Type: application/json*)
  - **Parámetros variable** codificados en la URL
  - (Opcionalmente) **Objeto JSON**
- **Servidor responde**
  - **Código resultado**. Ej.: 200 OK, 201 Created, 404 Not Found…
  - **Cabeceras** con opciones
  - (Opcionalmente) **Objeto JSON**

REST – Operaciones CRUD

- **CRUD**: operaciones elementales sobre una BD

Create



Read

Update

Delete

- Operaciones **CRUD mediante** métodos **HTTP**
  - **POST** *(Create)*
  - **GET** *(Read)*
  - **PUT** *(Update)* / **PATCH** *(Partial Update)*
  - **DELETE** *(Delete)*

Getting started with NGSI v2

Tutoriales FIWARE NGSI v2

Getting started with NGSI v2

- **Arquitectura**

- **Objetivos de aprendizaje**
  - Familiarizarse con el broker de contexto Orion
  - **Peticiones REST a Orion** mediante Postman

Getting started with NGSI v2
Datos de contexto

- **Entidad**
  - **Representación del estado un objeto real** del dominio de aplicación
  - **Atributos obligatorios**: ***id, type***
  - **Atributos opcionales**. Cada uno con atributos obligatorios ***type, value***
  - Ej.:

```
{
  "id": "urn:ngsi-ld:Store:001",
  "type": "Store",
  "address": {
    "type": "PostalAddress",
    "value": {
      "streetAddress": "Bornholmer Straße 65",
      "addressRegion": "Berlin",
      "addressLocality": "Prenzlauer Berg",
      "postalCode": "10439"
    },
    "metadata": {
      "verified": {
        "value": true,
        "type": "Boolean"
      }
    }
  },
  "location": {
    "type": "geo:json",
    "value": {
      "type": "Point",
      "coordinates": [13.3986, 52.5547]
    }
  },
  "name": {
    "type": "Text",
    "value": "Bösebrücke Einkauf"
  }
}
```

Getting started with NGSI v2
Líneas guía de uso del modelo

- Nombres de términos en **inglés americano**
- Nombres de **entidades empiezan por mayúscula**. Ej.: *Store*
- Nombres de **atributos en *camelCase***. Ej.: *postalCode*

- Los IDs de entidades deben ser un URN que siga las líneas guía de NGSI-LD: ***urn:ngsi-ld:<entity-type>:<entity-id>***
  - Consulte el concepto y los **ejemplos** de URN *(Uniform Resource Name)*
- Los **nombres** de tipos de datos deben reutilizar los de **Schema.org** siempre que sea posible

- Info de **localización** debe definirse mediante los **atributos *address* y *location***
- Usar **GeoJSON** para codificar las propiedades geoespaciales
- **Metadatos** de los atributos: nombres reservados
  - *dateCreated* (tipo *DateTime*): fecha creación del atributo (ISO 8601)
  - *dateModified* (tipo *DateTime*): fecha modif. del atributo (ISO 8601)
  - *previousValue* (tipo cualquiera): solo para notificaciones
  - *actionType* (tipo *Text*): solo para notificaciones

Getting started with NGSI v2
Acceso a los datos de contexto

- **Creación de entidades**
  - **POST /v2/entities**
    - *Headers: Content-Type: application/json*
    - *Body:* Representación JSON de la entidad
  - Respuesta:
    - *Status: 201 Created*
    - *Headers: Location:* URL de la entidad creada
- **Consulta de entidades**
  - **GET /v2/entities/**
    - Devuelve todas las entidades almacenadas
  - **GET /v2/entities/<id>?options=keyValues**
    - Devuelve la entidad con identificador <id>
    - En formato clave-valor (sin los atributos *type* y *metadata*)
  - **GET /v2/entities?type=<type>**
    - Devuelve todas las entidades de tipo <type>

Getting started with NGSI v2
Acceso a los datos de contexto (II)

- **Consulta de entidades con filtrado**
  - **GET /v2/entities?q=<query>**
    - **Filtrado mediante consulta *(query)***
    - Devuelve las entidades que cumplen la condición <query>.
    - La condición se expresa en el lenguaje Simple Query Language de Orion
    - Ej.: **Filtrado mediante comparación del valor de un atributo**

```
curl 'http://localhost:1026/v2/entities' \
    -d 'type=Store' \
    -d 'q=name==%27Checkpoint%20Markt%27' \
    -d 'options=keyValues'
```

    - Ej.: **Filtrado mediante la comparación del valor de un subatributo**

```
curl 'http://localhost:1026/v2/entities' \
    -d 'type=Store' \
    -d 'q=address.addressLocality==Kreuzberg' \
    -d 'options=keyValues'
```

Getting started with NGSI v2
Acceso a los datos de contexto (III)

- **Consulta de entidades con filtrado**
  - **GET /v2/entities?mq=<query>**
    - **Filtrado mediante consulta de metadatos *(metadata query)***
    - Devuelve en las entidades que cumplen la condición <mquery>.
    - La condición se expresa en el lenguaje Simple Query Language de Orion
    - Ej.:

```
curl 'http://localhost:1026/v2/entities' \
    -d 'type=Store' \
    -d 'mq=address.verified==true' \
    -d 'options=keyValues'
```

  - **GET /v2/entities?georel=<georel>&geometry=<g>&coordinates=<coords>**
    - **Filtrado mediante consulta de datos espaciales GeoJSON**
    - Devuelve las entidades cuyo atributo de tipo *geo:json* cumple la condición <georel>.
    - La condición se expresa mediante el lenguaje de consulta geoespacial de Orion
    - Ej.:

```
curl http://localhost:1026/v2/entities' \
  -d 'type=Store' \
  -d 'georel=near;maxDistance:1500' \
  -d 'geometry=point' \
  -d 'coords=52.5162,13.3777'
```

Entity Relationships

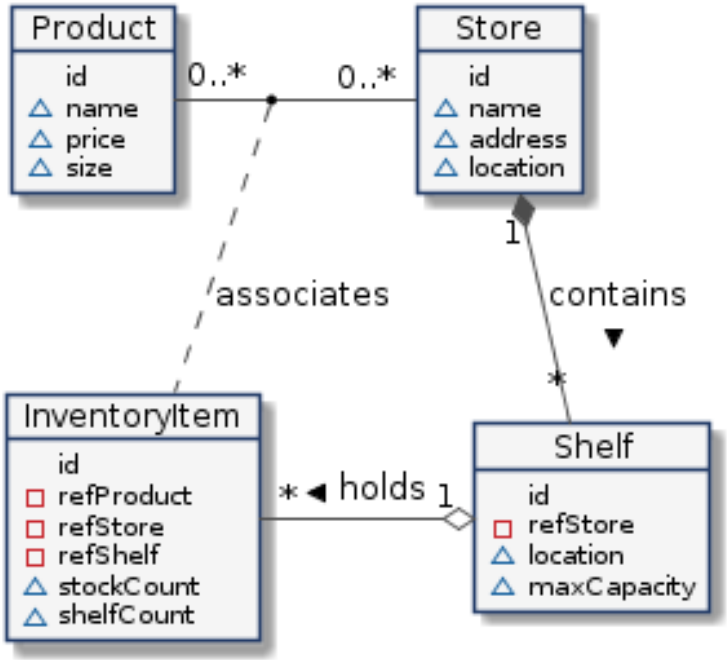
Tutoriales FIWARE NGSI v2

Entity Relationships

- **Arquitectura**

- **Objetivos de aprendizaje**
  - Entender y manejar las **relaciones entre entidades**

- **Modelo de datos**



Entity Relationships
Creación *batch* de entidades

- **Creación *batch* de entidades**
  - **POST /v2/op/update**
  - ***Body:***

    *{ **"actionType":** "append" | "update", **"entities":** [<JSON array>] }*
  - Ej.:

```
curl –X POST 'http://localhost:1026/v2/op/update' \
     -H 'Content-Type: application/json' \
     -d '{ "actionType":"append",
           "entities":[
              { "id":"urn:ngsi-ld:Shelf:unit001", "type":"Shelf",
                "location":{ "type":"geo:json",
                             "value":{ "type":"Point",
                                       "coordinates":[13.3986112, 52.554699]} },
                "name": { "type":"Text", "value":"Corner Unit"},
                "maxCapacity": {"type":"Integer", "value":50 }
              },
              { "id":"urn:ngsi-ld:Shelf:unit002", "type":"Shelf", ... },

              ...

              { "id":"urn:ngsi-ld:Shelf:unit005", "type":"Shelf", ... }
           ] }'
```

Entity Relationships
Creación *batch* de relaciones

- **Creación *batch* de relaciones**
  - Las **relaciones son también entidades**
  - Simplemente tienen **atributos *refEntityName* de tipo *Relationship*** que hacen referencia a otra entidad.
  - Ej.:

```
curl -X POST \
 'http://localhost:1026/v2/op/update' \
 -H 'Content-Type: application/json' \
 -d ‘{ "actionType":"append",
       "entities":[
          { "id":"urn:ngsi-ld:Shelf:unit001", "type":"Shelf",
            "refStore": { "type": "Relationship",
                          "value": "urn:ngsi-ld:Store:001" } },
          { "id":"urn:ngsi-ld:Shelf:unit002", "type":"Shelf",
            "refStore": { "type": "Relationship",
                          "value": "urn:ngsi-ld:Store:001" } }
       ] }'
```

Entity Relationships
Lectura de clave foránea de una relación

- **Dirección hija → madre**
  - Consulta del valor *(options=values)* del atributo *refEntityName* de la entidad hija
  - Ej.:
```
curl 'http://localhost:1026/v2/entities/urn:ngsi-ld:Shelf:unit001' \
  -d 'type=Shelf' \
  -d 'options=values' \
  -d 'attrs=refStore'
```
  - Resp.: `["urn:ngsi-ld:Store:001"]`

Entity Relationships
Lectura de clave foránea de una relación

- **Dirección madre → hija**
  - Consulta del atributo *refEntityName* de la entidad hija
  - Ej.:

```
curl 'http://localhost:1026/v2/entities' \
  -d 'q=refStore==urn:ngsi-ld:Store:001’ \
  -d 'type=Shelf’ \
  -d 'options=count' \
  -d 'attrs=type'
```

  - Resp.:

```
[  { "id": "urn:ngsi-ld:Shelf:unit001", "type": "Shelf" },
   { "id": "urn:ngsi-ld:Shelf:unit002", "type": "Shelf" },
   { "id": "urn:ngsi-ld:Shelf:unit003", "type": "Shelf"}  ]
```

  - Ej.:

```
curl 'http://localhost:1026/v2/entities' \
  -d 'q=refStore==urn:ngsi-ld:Store:001' \
  -d 'type=Shelf' \
  -d 'options=values' \
  -d 'attrs=name'
```

  - Resp.:

```
[ ["Corner Unit"], ["Wall Unit 1"], ["Wall Unit 2"] ]
```

Entity Relationships
Creación de relaciones N:M

- **Creación de relación N:M (tabla puente)**
  - **Cada entrada** de la tabla puente contiene **una clave foránea para cada entidad** que interviene en la relación N:M
  - Relaciona **dos o más entidades**
  - Ej.: Relación *InventoryItem* entre *Store*, *Shelf* y *Product*
    - Cada ítem de inventario representa la colocación de una(s) unidad(es) de un producto determinado en una balda concreta de un almacén

```
curl -X POST 'http://localhost:1026/v2/entities' \
-H 'Content-Type: application/json' \
-d '{
    "id": "urn:ngsi-ld:InventoryItem:001", "type": "InventoryItem",
    "refStore": { "type": "Relationship", "value": "urn:ngsi-ld:Store:001" },
    "refShelf": { "type": "Relationship", "value": "urn:ngsi-ld:Shelf:unit001" },
    "refProduct": { "type": "Relationship", "value": "urn:ngsi-ld:Product:001" },
    "stockCount":{ "type":"Integer", "value": 10000 },
    "shelfCount":{ "type":"Integer", "value": 50 }
}'
```

Entity Relationships
Consulta de relaciones N:M

- **Consulta de relación N:M (tabla puente)**
  - Ej.: **Productos que se venden en un almacén**

```
curl 'http://localhost:1026/v2/entities' \
  -d 'q=refStore==urn:ngsi-ld:Store:001' \
  -d 'options=values' \
  -d 'attrs=refProduct' \
  -d 'type=InventoryItem'}'
```

  - Resp.: `[["urn:ngsi-ld:Product:001"]]`

  - Ej.: **Almacenes que venden un producto**

```
curl 'http://localhost:1026/v2/entities' \
  -d 'q=refProduct==urn:ngsi-ld:Product:001' \
  -d 'options=values' \
  -d 'attrs=refStore'\
  -d 'type=InventoryItem'
```

  - Resp.: `[["urn:ngsi-ld:Store:001"]]`

Entity Relationships
Integridad Referencial

- **Orion NO proporciona integridad referencial**
- Las aplicaciones deben encargarse de garantizarla
- Ej.: antes de borrar un almacén, debemos comprobar que no tiene ninguna balda asociada

```
curl 'http://localhost:1026/v2/entities' \
  -d 'q=refStore==urn:ngsi-ld:Store:001' \
  -d 'options=count' \
  -d 'attrs=type'
```

CRUD Operations

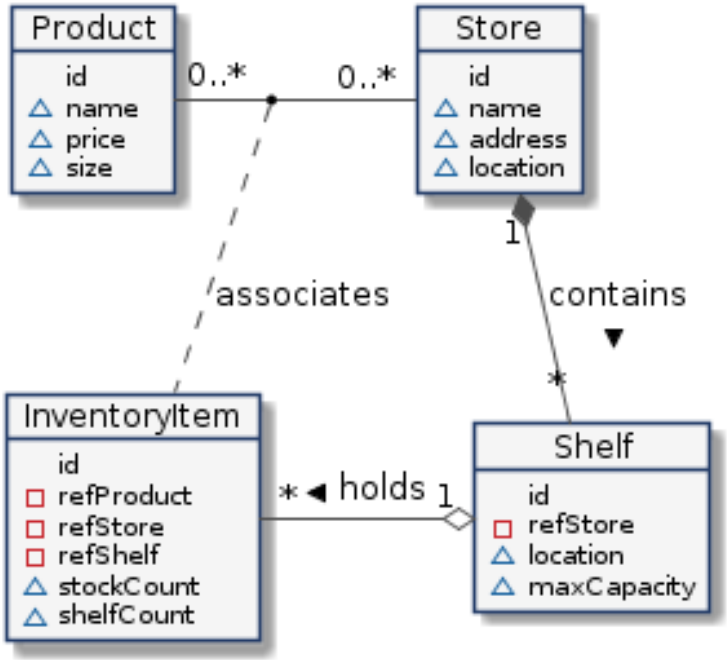
Tutoriales FIWARE NGSI v2

CRUD Operations

- **Arquitectura**

- **Objetivos de aprendizaje**
  - Manejar las **operaciones CRUD** en Orion

- **Modelo de datos**



CRUD Operations
Operaciones CRUD sobre entidades

- Especificación Swagger del API NGSIv2 sobre entidades

HTTP Verb	/v2/entities	/v2/entities/<entity-id>
POST	**CREA** una entidad	**CREA** o **ACTUALIZA** un atributo de una entidad
GET	**OBTIENE** todas las entidades Soporta filtrado	**OBTIENE** los datos de una entidad Soporta filtrado
DELETE		**BORRA** una entidad

CRUD Operations
Operaciones CRUD sobre atributos

- Especificación Swagger del API NGSIv2 sobre atributos

HTTP Verb	.../attrs	.../attrs/<attribute>	.../attrs/<attribute>/value
GET			**OBTIENE** el valor de un atributo de una entidad
PUT			**ACTUALIZA** el valor de un atributo de una entidad
PATCH	**ACTUALIZA** uno o varios atributos de una entidad		
DELETE		**BORRA** un atributo de una entidad	

CRUD Operations
Operaciones *batch* sobre entidades

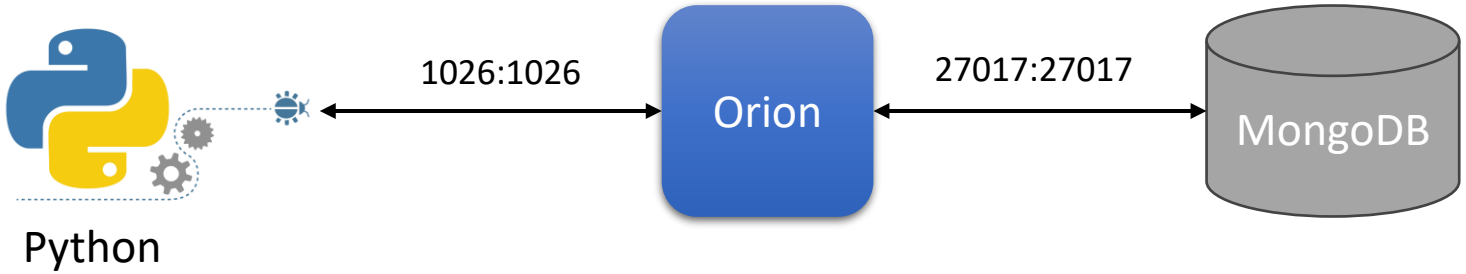
- **Operaciones *batch* sobre entidades**
  - **POST /v2/op/update**
  - ***Body:*** *{ **"actionType":***
    "append" | "append_strict" |
    "update" | "replace" |
    "delete",
    "entities": *[<JSON array>] }*
  - Especificación Swagger

Accessing Context

Tutoriales FIWARE NGSI v2

Accessing Context

- **Arquitectura**



- **Objetivos de aprendizaje**
  - Acceder programáticamente a **Orion desde Python (Tarea 1 de la Práctica 1)**

Accessing Context
Operaciones CRUD sobre entidades

- **Librería *ngsiv2.py* de operaciones CRUD sobre Orion**

HTTP Verb	/v2/entities	/v2/entities/<id>
POST	*status* = ***create_entity***(*entity*) **CREA** una entidad *entity* es de tipo *dict*	
GET	(*status, entity_list*) = ***list_entities***(*type = None,* *options = 'count', attrs = None*) **OBTIENE** la lista (*array* de *dicts*) de entidades almacenadas. Soporta filtrado.	(*status, entity*) = ***read_entity***(*id*) **OBTIENE** una entidad. Soporta filtrado *entity* es de tipo *dict*
DELETE		*status* = ***delete_entity***(*id*) **BORRA** una entidad

Accessing Context
Operaciones CRUD sobre atributos

HTTP Verb	.../attrs	.../attrs/<attribute>	.../attrs/<attribute>/value
GET			*(status, val) = **read_attr**(id, attr)* **OBTIENE** un atributo de una entidad
PUT			*status = **update_attr**(id, attr, val)* **ACTUALIZA** un atributo de una entidad
PATCH	*status = **update_attrs**(id, attrs_vals)* **ACTUALIZA** uno o más atributos de una entidad		
DELETE		*status = **delete_attr**(id, attr)* **BORRA** un atributo	

Accessing Context
Operaciones CRUD sobre entidades

- **Ejemplo: *read_entity()* y *create_entity()***

```
import requests
import json
base_path = 'http://localhost:1026/v2’

def read_entity(entity_id):
      r = requests.get(base_path + '/entities/' + entity_id)
      return (r.status_code, json.loads(r.text))

def create_entity(entity):
      r = requests.post(base_path + '/entities/', json = entity)
      return r.status_code
```

- **Uso**

```
import pprint
r = create_entity({‘id’: 'urn:ngsi-ld:Product:021’, ‘type’: 'Product’,
      'name': {'type': 'Text', 'value': 'Flower’},
      'price': {'type': 'Integer', 'value': 100},
      'size': {'type': 'String', 'value': 'XS'}})
pprint.pprint('Status code: ' + str(r))
(status, data) = read_entity('urn:ngsi-ld:Product:021')
pprint.pprint('Status code: ' + str(status))
if status == 200:
      pprint.pprint(data)
```

Accessing Context Consideraciones

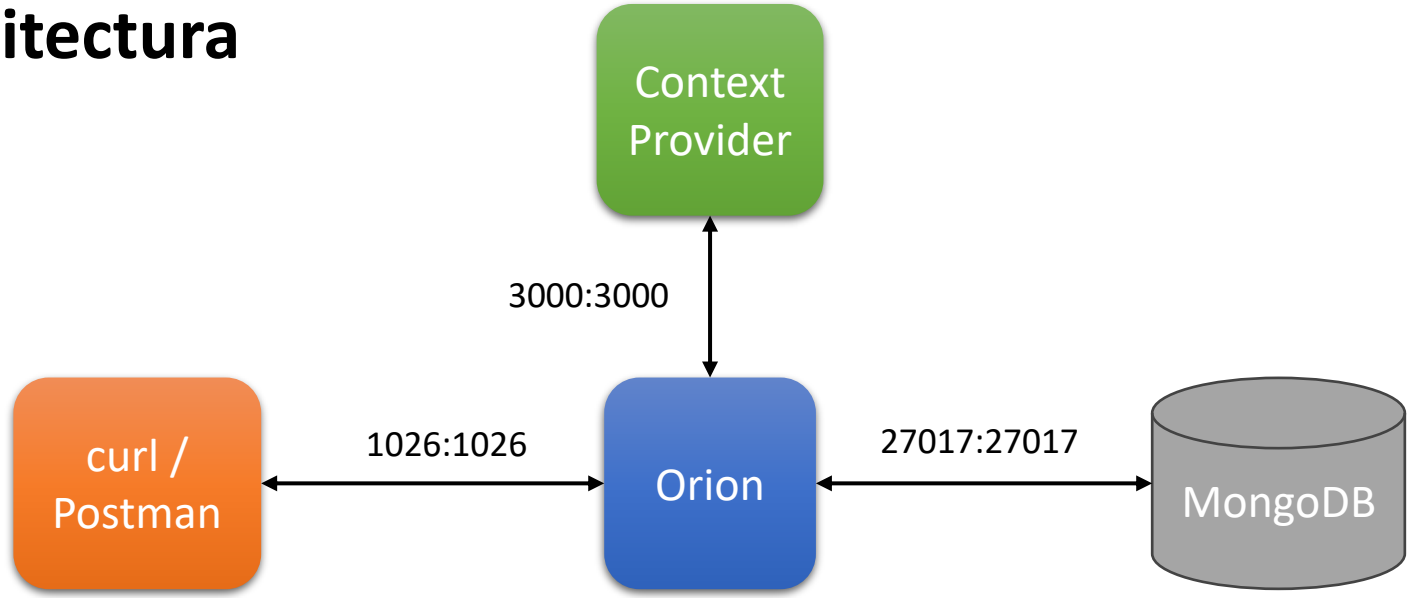
- Función ***update_attr()***
  - Petición **PUT** con *Content-Type = text/plain* en la cabecera
  - Ver formato de una petición similar en las colecciones de Postman y la documentación de la librería *requests*

Context Providers

Tutoriales FIWARE NGSI v2

Context Providers

- **Arquitectura**



- **Objetivos de aprendizaje**
  - Entender el papel de los **proveedores de contexto**

Context Providers

- Orion permite **registrar** proveedores de contexto
- Un **proveedor** de contexto **suministra info** de varios atributos de diversas entidades

- **Proveedores** de contexto del **tutorial**
  - Datos estáticos
  - Datos aleatorios
  - *Cat facts* (proxy de https://catfact.ninja/facts)

Context Providers
Suministro de datos desde el proveedor

- Orion **nunca almacena** los datos, siempre delega en el proveedor de contexto
- El proveedor de contexto debe implementar el *endpoint* NGSIv2 **POST /op/query** para que Orion pueda obtener datos
- Ej.:

```
curl -X POST 'http://localhost:3000/random/weatherConditions/op/query' \
  -H 'Cache-Control: no-cache' \
  -H 'Content-Type: application/json' \
  -d '{ "entities": [ { "type": "Store",
                        "isPattern": "false",
                        "id": "urn:ngsi-ld:Store:001" } ],
        "attrs": [ "temperature", "relativeHumidity" ] }'
```

  - Resp.:

```
[ { "id": "urn:ngsi-ld:Store:001",
    "type": "Store",
    "temperature": { "type": "Number", "value": 16 },
    "relativeHumidity": { "type": "Number", "value": 30 } } ]
```

Context Providers
Endpoint de **consulta** POST /op/query

- Orion también implementa el endpoint **POST /op/query**
  - Pese a ser POST, es para hacer consultas
  - Permite hacer **consultas que involucran entidades de distinto tipo** (no soportadas por GET /entities)

Context Providers
Registro de un proveedor de contexto

- Mediante **GET** / **POST** / **PATCH** / **DELETE** en ***/v2/registrations***
- Ej.: **Registro de un proveedor** de contexto

```
curl -X POST 'http://localhost:1026/v2/registrations' \
  -H 'Content-Type: application/json' \
  -d ‘{
  "description": "Random Weather Conditions",
  "dataProvided": {
       "entities": [ { "id": "urn:ngsi-ld:Store:001", "type": "Store" } ],
       "attrs": [ "temperature", "relativeHumidity“ ]
  },
  "provider": {
      "http": { "url": "http://context-provider:3000/random/weatherConditions" } }
  }'
```

- Orion invocará **POST** *<provider.http.url>**/op/query*** para obtener datos

Context Providers
Registro de un proveedor de contexto

- Ej.: **Listado de todos los proveedores** registrados

```
curl 'http://localhost:1026/v2/registrations'
```

  - Resp.:

```
[ { "id": "5ad5b9435c28633f0ae90671",
    "description": "Random Weather Conditions",
    "dataProvided": {
       "entities": [ { "id": "urn:ngsi-ld:Store:001", "type": "Store" } ],
       "attrs": ["relativeHumidity"]
    },
    "provider": {
       "http": { "url": "http://context-provider:3000/random/weatherConditions" },
       "supportedForwardingMode": "all",
       "legacyForwarding": false
    },
    "status": "active" } ]
```

- Ej.: **Des-registro de un proveedor**

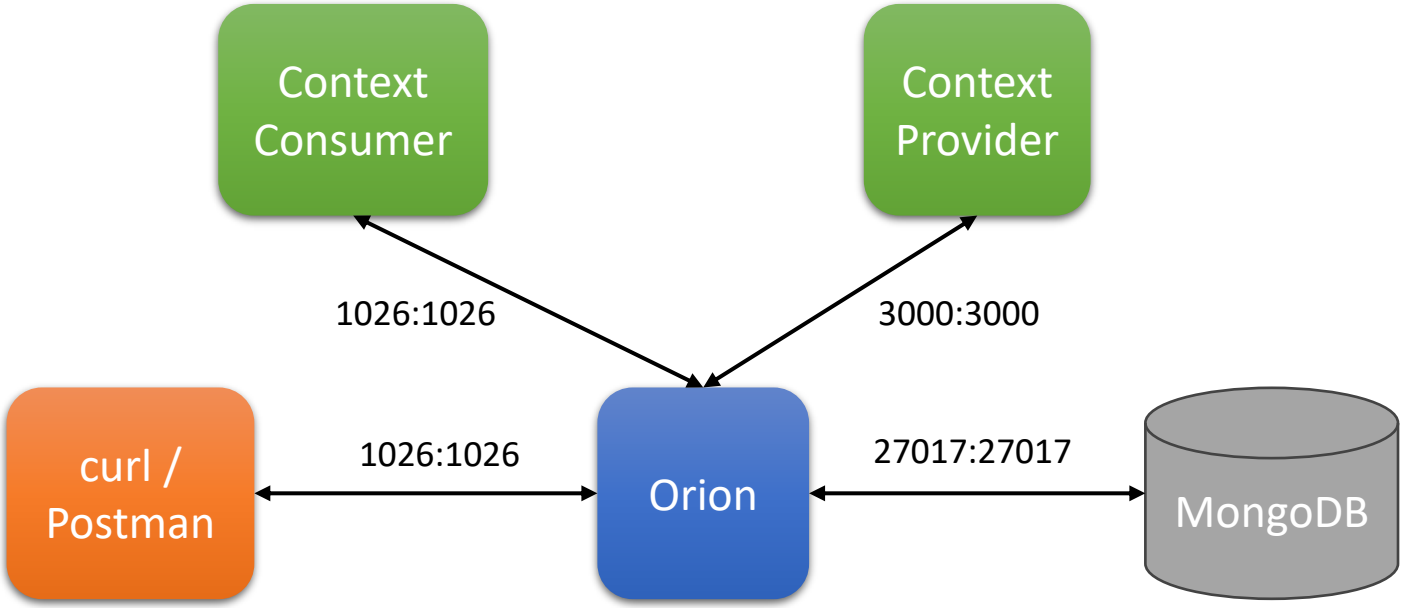
```
curl -X DELETE \
     'http://localhost:1026/v2/registrations/5ad5b9435c28633f0ae90671'
```

Subscriptions

Tutoriales FIWARE NGSI v2

Subscriptions

- **Arquitectura**



- **Objetivos de aprendizaje**
  - **Manejo de suscripciones** por parte de los consumidores de contexto

Suscriptions

- **Orion avisa a sus suscriptores de los cambios** en el contexto
- En Orion invocamos **GET** / **POST** / **PATCH** / **DELETE** **/v2/subscriptions** para **gestionar las suscripciones**
- Ej.: **Creación de una suscripción**

```
curl -X POST 'http://localhost:1026/v2/subscriptions' \
  -H 'content-type: application/json' \
  -d '{ "description": "Notify me of all product price changes",
        "subject": { "entities": [{"idPattern": ".*", "type": "Product"}],
                     "condition": { "attrs": [ "price" ] } },
        "notification": {
            "http": { "url": "http://tutorial:3000/subscription/price-change" } }
      }'
```

Suscriptions Optimizaciones

- **Reducción de la carga útil**
  - ***attrs:*** atributos que nos interesa recibir
  - ***attrsFormat:*** *keyValues* | *values*
- **Reducción del alcance** mediante ***expression***
- Ej.:

```
curl -X POST 'http://localhost:1026/v2/subscriptions' \
    -H 'Content-Type: application/json' \
    -d ‘{
    "description": "Notify me of low stock in Store 001",
    "subject": {
       "entities": [{"idPattern": ".*","type": "InventoryItem"}],
       "condition": {
           "attrs": ["shelfCount"],
           "expression": {"q": "shelfCount<10;refStore==urn:ngsi-ld:Store:001"}
       }
    },
    "notification": {
       "http": { "url": "http://tutorial:3000/subscription/low-stock-store001" },
       "attrsFormat" : "keyValues"
    }
    }'
```